Topic 10: Protocols and Extensions

Task 1: Payroll with Protocols

```
protocol Payable {
    var name: String { get }
    func monthlyPay() -> Double
}

extension Payable {
    func payslip() -> String {
        return "\(name)'s monthly pay: Rs. \(monthlyPay())"
    }
}
struct FullTimeEmployee: Payable {
    var name: String
    var annualSalary: Double

    func monthlyPay() -> Double {
        return annualSalary / 12
    }
}

struct Intern: Payable {
    var name: String
    var monthlyStipend: Double

    func monthlyPay() -> Double {
        return monthlyStipend
    }
}

struct Contractor: Payable {
    var name: String
    var hourlyRate: Double
    var hours: Double

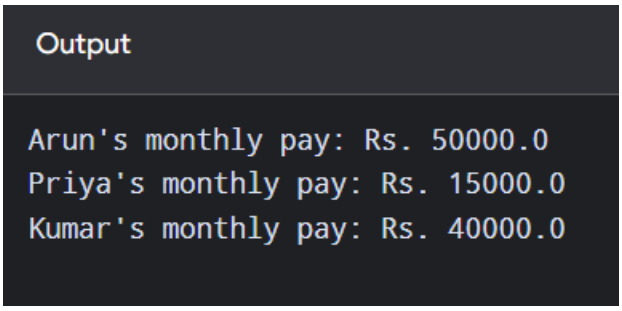
    func monthlyPay() -> Double {
        return hourlyRate * hours
    }
}

let payroll: [Payable] = [
    FullTimeEmployee(name: "Arun", annualSalary: 600000),
```

```
    Intern(name: "Priya", monthlyStipend: 15000),
    Contractor(name: "Kumar", hourlyRate: 500, hours: 80)
]

for person in payroll {
    print(person.payslip())
}
```

OUTPUT:



Task 2: Extend Existing Types

```
import Foundation
extension Int {
    var isEven: Bool {
        return self % 2 == 0
    }

    func squared() -> Int {
        return self * self
    }
}

print(4.isEven)
print(5.isEven)
print(3.squared())

extension Array where Element == Int {
    var sum: Int {
        return self.reduce(0, +)
    }
}
print([1, 2, 3, 4].sum)

extension String {
    var wordCount: Int {
        return self.split { $0.isWhitespace }.count
```

```
    }
}
print("Swift is easy".wordCount)

struct Version: Comparable {
    var major: Int
    var minor: Int

    static func < (lhs: Version, rhs: Version) -> Bool {
        if lhs.major != rhs.major {
            return lhs.major < rhs.major
        }
        return lhs.minor < rhs.minor
    }
}

let v1 = Version(major: 1, minor: 9)
let v2 = Version(major: 2, minor: 0)

print(v1 < v2)

let versions = [
    Version(major: 2, minor: 1),
    Version(major: 1, minor: 9),
    Version(major: 1, minor: 5)
]

let sortedVersions = versions.sorted()

for version in sortedVersions {
    print("\(version.major).\(version.minor)")
}
```

OUTPUT:

Output

```
true
false
9
10
3
true
1.5
1.9
2.1
```

Task 3: Protocol Composition Over Inheritance

```
protocol Flyable {
    func fly()
}

protocol Swimmable {
    func swim()
}

struct Duck: Flyable, Swimmable {
    func fly() {
        print("Duck is flying")
    }

    func swim() {
        print("Duck is swimming")
    }
}

struct Eagle: Flyable {
    func fly() {
        print("Eagle is flying")
    }
}

struct Fish: Swimmable {
    func swim() {
        print("Fish is swimming")
    }
```

```
}

func showOff(_ animal: Flyable & Swimmable) {
   animal.fly()
   animal.swim()
}

let duck = Duck()
let eagle = Eagle()
let fish = Fish()

eagle.fly()
fish.swim()

showOff(duck)
```

OUTPUT:

